AGRICULTURE INFORMATION SYSTEM

A SUMMER INTERNSHIP REPORT

Submitted by

LAVANYA D– 810023205089



.

DEPARTMENT OF INFORMATION TECHNOLOGY

ANNA UNIVERSITY ,

TIRUCHIRAPALLI – 620 024.

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS),

ANNA UNIVERSITY , TIRUCHIRAPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify this Summer Internship project report **"Agriculture Information System"** is the Bonafide work of **LAVANYA D (810023205089)** has satisfactory completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2025-2026.**

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on _____.

ABSTRACT

The Agriculture Information System is a full-stack web application, developed under the name “GreenPath,” built to give farmers and agricultural department officials a single digital platform for the information they need to run a farm efficiently. Through the platform, farmers can register and log in, look up crop information such as soil type, fertilizer, and irrigation guidance, check weather forecasts for their location, view and save market prices for crops in their area, maintain farm records covering land and production details, and browse government schemes and benefits relevant to their farming operations. The system also offers a reports and analytics view that summarises a farmer's saved data at a glance.

The project is implemented as a MERN-style full-stack application. The frontend is built with React and Vite, styled with a clean, card-based responsive layout, giving users a fast and interactive interface for browsing features and entering data. The backend is developed using Node.js and Express, exposing REST APIs for authentication, crop information, weather data, market prices, farm records, and government schemes. MongoDB is used as the database, with separate collections for applications, crops, farm records, market prices, and schemes, allowing flexible and scalable storage of agricultural data.

One of the main objectives of this system is to reduce the difficulty farmers face in accessing timely, accurate agricultural information — replacing scattered, informal sources with a centralized, always-available digital resource. By digitizing crop guidance, weather forecasts, market price tracking, farm record-keeping, and government scheme discovery, the system helps farmers make better-informed decisions, track their own farm data over time, and identify government support they may be eligible for.

Overall, the Agriculture Information System demonstrates the practical application of full-stack development principles using React, Node.js, Express, and MongoDB, and highlights how modern web technology can be used to support farmers with real-time agricultural knowledge.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
1	INTRODUCTION 1.1 OBJECTIVE OF THE INTERNSHIP	1 - 8
2	COMPANY PROFILE	8 - 11
3	DAILY WORK REPORT	12 - 15
4	BLOCK DIAGRAM	16
5	TRAVEL AGENCY MANAGEMENT SYSTEM 5.1 EXISTING SYSTEM 5.2 PROPOSED SYSTEM 5.3 CAPABILIITIES	16 - 19
6	PROPOSED METHOD 6.1 User Authentication 6.2 Crop Information Management 6.3 Weather Forecast Module 6.4 Market Price Management 6.5 Farm Records Management 6.6 Government Schemes Module 6.7 Reports & Analytics	20 - 22
7	ALGORITHM 7.1 User Login 7.2 User Registration 7.3 Crop Information Retrieval 7.4 Weather Forecast Fetching 7.5 Farm Record Management	21 - 24
8	CODE IMPLEMENTATION	24- 29
9	SAMPLE OUTPUT	30 - 33
10	ANALYSIS	34
11	CONCLUSION	35
12	FUTURE WORK	35 - 36
13	REFERENCES	36

1.INTRODUCTION

The Agriculture Information System, developed under the product name "GreenPath," is a full-stack web application built to give farmers and agricultural officials easy, centralized access to the information they need to run a farm well. Farmers across Tamil Nadu commonly rely on scattered sources — local shopkeepers, word of mouth, printed notices, and informal networks — for crop guidance, market prices, and information on government schemes. This makes it difficult to get timely, accurate, and location-specific information, and often leads farmers to miss beneficial schemes or make decisions based on outdated price information.

The primary purpose of this project is to provide a secure, easy-to-use platform where farmers can register, log in, look up crop and soil information, check weather forecasts for their location, view current market prices for crops in nearby markets, maintain their own farm records, and explore government schemes relevant to them. The system also gives administrators and department officials a way to manage the underlying data — crop information, market prices, and scheme listings — so that what farmers see stays current and reliable.

The application is built using modern full-stack JavaScript technologies. The frontend is developed with React and Vite, producing a fast, responsive, card-based interface that works well on both desktop and mobile screens. The backend is developed using Node.js with the Express framework, which exposes REST APIs for authentication, crop data, weather data, market prices, farm records, and government schemes. MongoDB is used as the database, with the data organised into collections — crops, farmrecords, marketprices, schemes, and applications — that map naturally onto the different types of agricultural information the system manages.

The system follows a clear client-server architecture: a Presentation Layer (the React frontend), an Application Layer (the Express REST API and business logic), and a Data Layer (MongoDB). The frontend communicates with the backend over HTTP using REST APIs, which keeps the interface responsive while all persistent data is safely stored and managed on the server side.

The Agriculture Information System is organised into modules including User Registration and Login, Crop Information, Weather Forecast, Market Prices,

Farm Records, Government Schemes, and Reports & Analytics. Each module addresses a specific need of a farmer or department official, and together they form a complete, practical agricultural information platform.

Overall, this project is a practical implementation of full-stack development using React, Node.js, Express, and MongoDB, and it provided hands-on experience in frontend design, REST API development, database modelling, and building software intended to be genuinely useful to farmers. The system can be extended in future with features such as AI-based crop recommendations, SMS alerts, and multilingual support to widen its reach among farming communities.

1.1 OBJECTIVES OF THE INTERNSHIP

The main objective of the Agriculture Information System is to develop a secure, easy-to-use web application, using React, Node.js, Express, and MongoDB, that gives farmers centralized access to agricultural information and gives department officials a way to manage that information.

The objectives of the project are:

- To develop a full-stack web application using React, Node.js, Express, and MongoDB.
- To provide secure registration and login for farmers and department officials.
- To allow farmers to look up and save crop, soil, fertilizer, and irrigation information.
- To provide weather forecasts for a farmer's location.
- To let farmers view and save market prices for crops in their area.
- To allow farmers to maintain farm records covering land and production details.

- To help farmers discover and explore relevant government schemes and benefits.
- To provide a reports and analytics summary of a farmer's saved data.
- To store agricultural data securely and reliably in a MongoDB database.
- To develop a responsive, reliable, and easy-to-use application accessible on both desktop and mobile devices.

1.2 MOTIVATION

The motivation behind the Agriculture Information System is to make essential agricultural knowledge easier for farmers to reach. Many farmers currently depend on informal, inconsistent sources for crop advice, current market prices, and information about government schemes, which can lead to missed opportunities and poorly-timed decisions.

This project provides an online platform where farmers can look up crop and soil guidance, check the weather, track market prices, record their own farm activity, and browse government schemes, all from one place. It also gives department officials a system through which they can keep this information current for the farming community they serve.

A further motivation was to gain practical, hands-on experience of full-stack JavaScript development — building a responsive React frontend, a Node.js/Express REST API, and a MongoDB data layer — while working on a project with genuine social and economic relevance to the agricultural sector.

2.COMPANY PROFILE



About the Company:

Astonish InfoTech Private Limited is a leading **Information Technology (IT)** company located **in Tiruchirappalli (Trichy), Tamil Nadu, India**. The company specializes in providing high-quality software solutions, web application development, mobile application development, and digital technology services to clients from various business sectors. Established on **25 February 2010**, the company has built its reputation by delivering reliable, innovative, and customer-oriented software products that meet modern business requirements.

The primary objective of **Astonish InfoTech** is to develop software solutions that improve productivity, simplify operations, and provide user-friendly digital services. The company follows modern software development methodologies and focuses on delivering scalable, secure, and efficient applications using the latest technologies, with an emphasis on quality assurance, customer satisfaction, and continuous technological improvement.

Apart from software development, the company actively provides **industrial training and internship programs** for engineering students and fresh graduates, designed to bridge the gap between academic learning and industry requirements through hands-on exposure to real-time projects. During the internship, students gain practical experience in software development, database management, frontend and backend integration, REST API development, software testing, and project deployment under the guidance of experienced professionals.

As part of my internship **at Astonish InfoTech Private Limited**, I developed the **Agriculture Information System** ("**GreenPath**") **using React, Node.js, Express, and MongoDB**. The project involved designing a responsive user interface, developing **backend REST services**, integrating the **MongoDB database,** and testing the application end to end. This internship helped me

improve my technical knowledge, programming skills, problem-solving ability, and understanding of the complete Software Development Life Cycle (SDLC).

The internship experience also provided practical exposure to working in a professional software development environment — understanding requirements, building a real-world application, using modern development tools, and following industry-standard practices while building the solution. Overall, Astonish InfoTech Private Limited provided an excellent platform to apply theoretical concepts in practical situations and strengthen my skills in **Mern Stack Development.**

Company Details:

- **Company Name:** Astonish InfoTech Private Limited
- **Established:** 25 February 2010
- **Industry:** Information Technology (IT) Services and Software Development
- **Location:** Technology Park (BUTP), Bharathidasan University Campus, Kajamalai, Tiruchirappalli, Tamil Nadu
- **Development Technologies:** Java, Spring Boot, HTML, CSS, JavaScript, Bootstrap, MySQL, REST APIs
- **Major Services:** Software Development, Web Development, Mobile Application Development, Database Solutions, Digital Marketing, Web Hosting, Technical Training
- **Internship Training:** Java Full Stack Development, Web Application Development, Database Management, Software Testing, Real-Time Projects

- **Development Approach:** Customer-oriented, scalable, secure, and industry-standard software solutions
- **Project Completed During Internship:** Travel Agency Management System (Java Full Stack)

Services Offered

- Software Development
- Web Application Development
- Website Design and Development
- Mobile Application Development
- Java Full Stack Development
- Database Design and Management
- REST API Development
- Digital Marketing and SEO
- Domain Registration and Web Hosting
- Technical Internship and Industrial Training
- Software Maintenance and Technical Support

Technologies Used by the Company

- **Programming Languages:** Java, JavaScript
- **Frontend:** HTML5, CSS3, Bootstrap
- **Backend:** Java Spring Boot
- **Database:** MySQL
- **Development Tools:** Eclipse, IntelliJ IDEA, VS Code

- **Build Tool:** Apache Maven
- **Version Control:** Git and GitHub
- **API Technology:** RESTful Web Services

3. DAILY WORK REPORT

Day 1: Attended orientation session; briefed on internship guidelines, evaluation criteria, and reporting format. Discussed and finalized the project topic – "Agriculture Information System" – with the faculty in-charge.

Day 2: Conducted requirement gathering by discussing with the mentor. Listed the stakeholders (Farmer, Department Official/Admin) and identified the core functional requirements of the agriculture information platform.

Day 3: Carried out a literature review of existing agriculture information portals and apps. Studied their features and documented the drawbacks of manual, word-of-mouth based access to crop and market information.

Day 4: Finalized the scope and objectives of the project. Prepared the list of hardware and software requirements needed for development.

Day 5: Set up the development environment – installed Node.js, npm, VS Code, and MongoDB Compass. Verified installation with a sample React app scaffolded using Vite.

Day 6: Studied React fundamentals – components, props, state, hooks (useState, useEffect), and React Router for page navigation.

Day 7: Designed the overall system architecture of the application using a layered approach (Presentation Layer, Application/API Layer, and Data Layer). Prepared the architecture (block) diagram.

Day 8: Designed the MongoDB database schema by identifying collections – Users/Applications, Crops, Farm Records, Market Prices, and Schemes – along with their fields.

Day 9: Prepared the Entity-Relationship diagram showing relationships between User, Farm Record, Crop, Market Price, and Scheme data.

Day 10: Created wireframes for the Home, Login, and Registration pages. Planned the responsive card-based layout for the features grid.

Day 11: Developed the Home Page using React, including the navigation bar, hero banner ("Empowering Farmers with Real-Time Agricultural Knowledge"), statistics section, and footer.

Day 12: Developed the Login Page UI with an email/password form and client-side validation.

Day 13: Developed the Registration Page UI with fields for name, email, phone, role selection (Farmer / Department Official), and password confirmation, along with input validations.

Day 14: Initialized the Node.js backend project using Express. Organized the project into routes, controllers, models, and config folders, and connected it to MongoDB.

Day 15: Created the Mongoose schema/model files – User, Crop, and Farm Record – defining their fields and validation rules.

Day 16: Implemented the authentication routes and controller to handle new user registration, including input validation and duplicate-email checks.

Day 17: Implemented the login route and controller for the authentication module, including password verification logic.

Day 18: Tested the Registration and Login REST APIs using Postman. Verified success and error responses (e.g., invalid credentials, duplicate user).

Day 19: Developed the Crop Information module – Crop model, crop routes, and crop controller – to support adding, updating, and listing crop, soil, fertilizer, and irrigation details.

Day 20: Developed the Weather Forecast module, integrating a weather API to fetch and save forecasts for a farmer's saved location.

Day 21: Built the Crop Information and Weather Forecast pages on the frontend and connected them to the respective backend APIs using fetch/Axios calls.

Day 22: Developed the Market Prices module – Market Price model, market price routes, and controller – to record crop name, market name, price, and date.

Day 23: Designed and integrated the Market Prices page on the frontend, allowing farmers to view and save current prices for crops in their area.

Day 24: Developed the Farm Records module – Farm Record model, routes, and controller – to let farmers record land and production details.

Day 25: Designed the Farm Records page frontend, allowing farmers to add, view, and update their farm records, and integrated it with the REST API.

Day 26: Developed the Government Schemes module, enabling farmers to browse and save government schemes and benefits relevant to their farming operations.

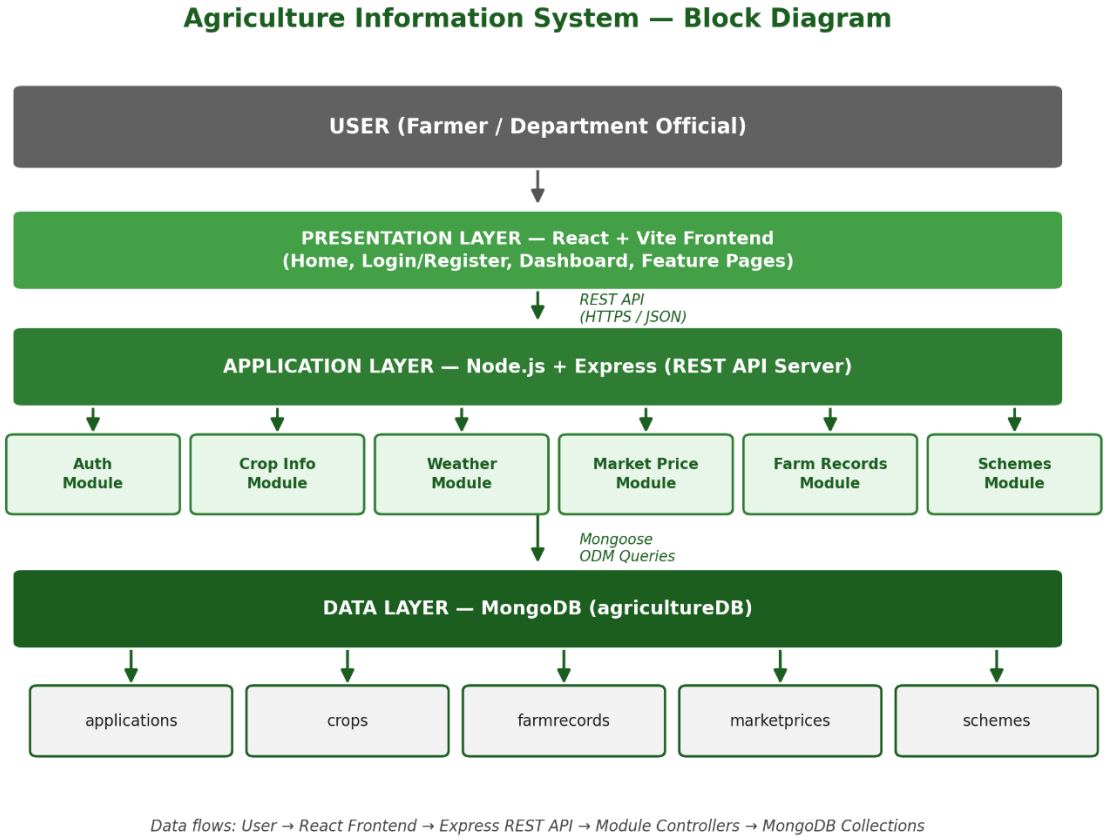
Day 27: Developed the Reports & Analytics module to summarise a farmer's saved crop, market price, and farm record data.

Day 28: Developed the Admin-side management screens for department officials to add, update, and manage crop information, market prices, and scheme listings.

Day 29: Verified and reviewed the MongoDB collections (applications, crops, farm records, market prices, schemes) using MongoDB Compass to confirm data was being stored correctly across modules.

Day 30: Performed testing of all modules, fixed identified bugs, took screenshots for documentation, and compiled the final internship report.

4.BLOCK DIAGRAM



5. AGRICULTUR INFORMATION SYSTEM

5.1 Existing System

Most farmers currently depend on informal and inconsistent sources for the information they need to run their farms. Crop and soil guidance is often obtained by word of mouth from other farmers or local shopkeepers; weather information is checked through generic apps not tailored to a specific plot of land; market prices are learned by visiting or calling the local market directly; and awareness of government schemes usually spreads slowly through informal networks.

This existing approach suffers from several limitations:

- **Scattered information sources** – crop guidance, weather, prices, and scheme details are spread across different, often unreliable channels
- **No centralized record-keeping** – farmers have no easy way to keep a running record of their land and production details.
- **Delayed market information** – prices are learned only by visiting or calling markets directly, causing farmers to sell at outdated prices.
- **Missed government schemes –** without a central place to browse schemes, eligible farmers often miss out on benefits they qualify for.
- **Limited accessibility** – information is typically available only during market hours or through in-person visits, not anytime, anywhere.
- **No analytics** – farmers have no simple way to see a summary of their own farm data over time.

These drawbacks make the existing situation inadequate for supporting today's farmers, and highlight the need for a digital, centralized agricultural information platform.

5.2 Proposed System

The proposed Agriculture Information System ("GreenPath") is a full-stack web application that centralizes the core information farmers need, in one accessible platform. It replaces scattered, informal sources with an integrated online system available to farmers and department officials at any time.
The system is built using React for the frontend and Node.js with Express and MongoDB for the backend, providing a responsive, secure, and scalable solution. Key aspects of the proposed system include:

- **Centralized platform** – crop, weather, market price, farm record, and government scheme data are all accessible from a single system.
- **Self-service farmer portal** – farmers can register, log in, look up crop information, check weather, view market prices, and manage their own farm records independently.
- **Real-time weather and market data** – farmers can check up-to-date weather forecasts and market prices for their location.

- **Digital farm record-keeping** – farmers can maintain land and production records that were previously kept, if at all, on paper.

- **Government scheme discovery** – farmers can browse and explore schemes and benefits relevant to them.

- **Reports & analytics** – farmers get a summary view of their saved data to support decision-making.

- **Admin/official management** – department officials can keep crop information, market prices, and scheme listings current.

By digitizing these processes, the proposed system reduces the effort needed to access reliable agricultural information and helps farmers make better-informed, timelier decisions

5.3 Capabilities

The Agriculture Information System offers the following key capabilities:

- **User Management** – secure registration, login, and profile management for farmers and department officials.

- **Crop Information** – get and save details about crops, soil, fertilizer, and irrigation.

- **Weather Forecast** – check and save weather forecasts for the farmer's location.

- **Market Prices** – save and view market prices for crops in the farmer's area.

- **Farm Records** – maintain farm records including land, production, and more.

- **Government Schemes** – save and explore government schemes and benefits.

- **Reports & Analytics** – view a summary and analytics of the farmer's saved data.

- **Scalability** – built on a layered Node.js/Express architecture, supporting future growth in data, users, and integrations.

Together, these capabilities make the system a comprehensive, reliable, and farmer-friendly solution for accessing real-time agricultural knowledge.

6. PROPOSED METHOD

6.1 User Authentication

- Handles registration and login for farmers and department officials
- Captures name, email, phone, password, and role during sign-up
- Validates credentials and prevents duplicate email registrations
- Grants role-based access: farmers → farmer dashboard, officials → admin panel
- Displays clear error messages for invalid login attempts

6.2 Crop Information Management

- Manages crop details: soil type, fertilizer recommendations, and irrigation guidance
- Admin/officials can add and update crop information
- Farmers can browse and search crop information relevant to their farm
- Uses a Crop model, crop routes, and crop controller

6.3 Weather Forecast Module

- Fetches and displays weather forecasts for a farmer's saved location
- Allows farmers to save forecasts for later reference
- Integrates with an external weather data source through the backend
- Displays temperature, conditions, and location-specific alerts

6.4 Market Price Management

- Manages crop, market name, price, and date details
- Admin/officials can add and update market price entries
- Farmers can browse market prices for crops in their area
- Uses a Market Price model, market price routes, and controller

6.5 Farm Records Management

- Allows farmers to record land size, crops grown, and production details
- Farmers can add, view, and update their own farm records
- Records are stored per farmer for later reference and reporting
- Uses a Farm Record model, farm record routes, and controller

6.6 Government Schemes Module

- Lists government schemes and benefits relevant to farmers
- Admin/officials can add and update scheme listings
- Farmers can browse, search, and save schemes they are interested in
- Uses a Scheme model, scheme routes, and controller

6.7 Reports & Analytics

- Summarises a farmer's saved crop, farm record, and market price data
- Presents summary statistics and simple analytics for the farmer's dashboard
- Helps farmers track their own data over time
- Supports administrators in monitoring overall platform usage

7. ALGORITHM

7.1 User Login

- User enters email and password on the login form
- System checks both fields are filled before submitting
- Credentials sent to the authentication route → auth controller
- System checks if the email exists in the database
- System compares the entered password with the stored (hashed) password
- If either check fails → return “Invalid Email or Password”
- If valid → redirect based on role (Farmer → dashboard, Official → admin panel)

7.2 User Registration

- User fills in name, email, phone, password, and role
- System validates all fields are filled and the password meets strength rules
- Data sent to the users route → user controller
- System checks if the email is already registered
- If duplicate → reject with "Email already registered"
- If unique → create and save the new user document via the User model
- Redirect the user to the login page on success

7.3 Crop Information Retrieval

- Farmer searches or browses for a crop on the Crop Information page
- Request sent to the crop route → crop controller
- System queries the Crop collection in MongoDB
- If a match is found → return soil, fertilizer, and irrigation details
- If no match is found → return an empty result with a suggestion message
- Farmer can optionally save the crop information for later reference

7.4 Weather Forecast Fetching

- Farmer selects or confirms their location on the Weather Forecast page
- Request sent to the weather route → weather controller
- Backend fetches forecast data from the external weather source
- System formats and returns temperature, conditions, and outlook to the frontend
- Farmer can save the forecast for their records
- If the external fetch fails → return a clear error message to the frontend

7.5 Farm Record Management

- Farmer opens the Farm Records page and chooses to add or update a record
- Farmer enters land size, crop, and production details and submits the form
- Request sent to the farm records route → farm record controller
- System validates the input fields
- If valid → save/update the record via the FarmRecord model in MongoDB
- Updated record list is returned to the frontend and displayed to the farmer

8. CODE IMPLEMENTATION

8.1 Authentication Module Code

Contains authentication logic for validating user credentials.

```
// authController.js
const User = require('../models/User');
const bcrypt = require('bcryptjs');

exports.login = async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user) {
    return res.status(401).json({ message: 'Invalid Email or Password' });
  }
  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) {
    return res.status(401).json({ message: 'Invalid Email or Password' });
  }
```

```
  return res.status(200).json({ message: 'Login Successful', role: user.role
});
};
```

8.2 Registration Module Code

Handles new farmer/official account creation and validation.

```
// userController.js
const User = require('../models/User');
const bcrypt = require('bcryptjs');

exports.registerUser = async (req, res) => {
  const existing = await User.findOne({ email: req.body.email });
  if (existing) {
    return res.status(400).json({ message: 'Email already registered' });
  }
  const hashed = await bcrypt.hash(req.body.password, 10);
  const newUser = await User.create({ ...req.body, password: hashed });
  return res.status(201).json(newUser);
};
```

8.3 Crop Information Module Code

Manages crop, soil, fertilizer, and irrigation information.

```
// cropController.js
const Crop = require('../models/Crop');

exports.getCrops = async (req, res) => {
  const crops = await Crop.find();
  res.status(200).json(crops);
};

exports.addCrop = async (req, res) => {
```

```
  const crop = await Crop.create(req.body);
  res.status(201).json(crop);
};
```

PaymentService.java @Service

```
public class PaymentService {
   @Autowired
   private PaymentRepository paymentRepository;
   public Payment makePayment(Payment payment) {
      payment.setStatus("SUCCESS");
      payment.setPaymentDate(LocalDate.now());
      return paymentRepository.save(payment);
   }
}
```

8.4 Market Price Module Code

Processes market price entries and retrieval.

```
// marketPriceController.js
const MarketPrice = require('../models/MarketPrice');


exports.addMarketPrice = async (req, res) => {
  const { cropName, marketName, price, date } = req.body;
  const entry = await MarketPrice.create({
    cropName, marketName, price, date
  });
  res.status(201).json(entry);
};
```

```
exports.getMarketPrices = async (req, res) => {
  const prices = await MarketPrice.find().sort({ date: -1 });
  res.status(200).json(prices);
};
```

8.5 Farm Records Module Code

Provides operations for creating and retrieving a farmer's farm records.

```
// farmRecordController.js
const FarmRecord = require('../models/FarmRecord');

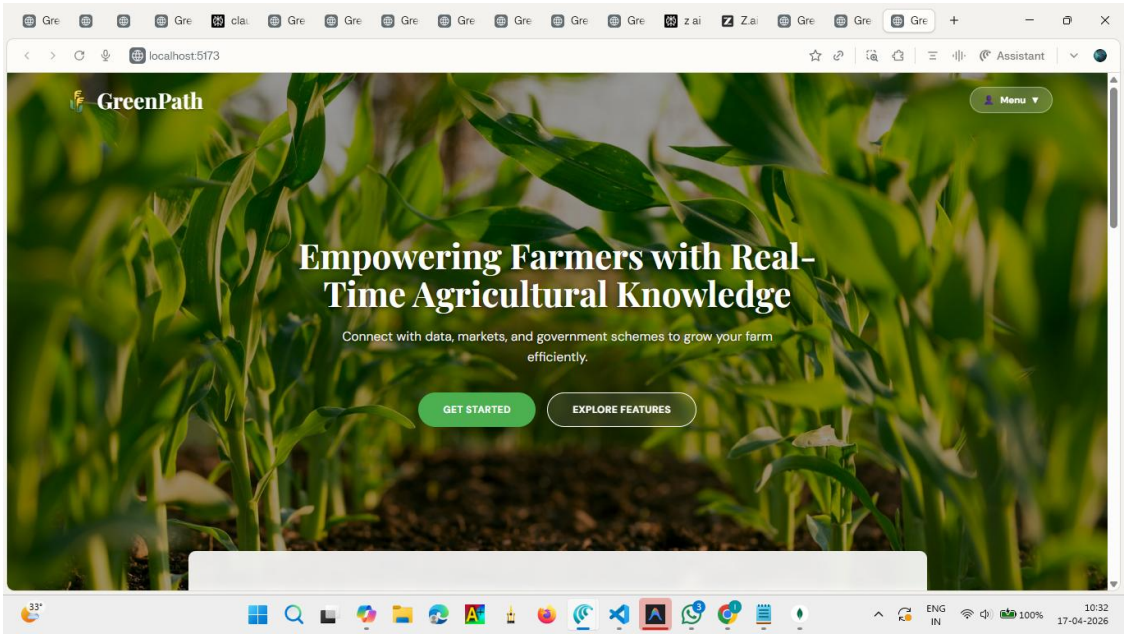
exports.addFarmRecord = async (req, res) => {
  const record = await FarmRecord.create(req.body);
  res.status(201).json(record);
};

exports.getFarmRecords = async (req, res) => {
  const records = await FarmRecord.find({ userId: req.params.userId });
  res.status(200).json(records);
};
```

9. SAMPLE OUTPUT

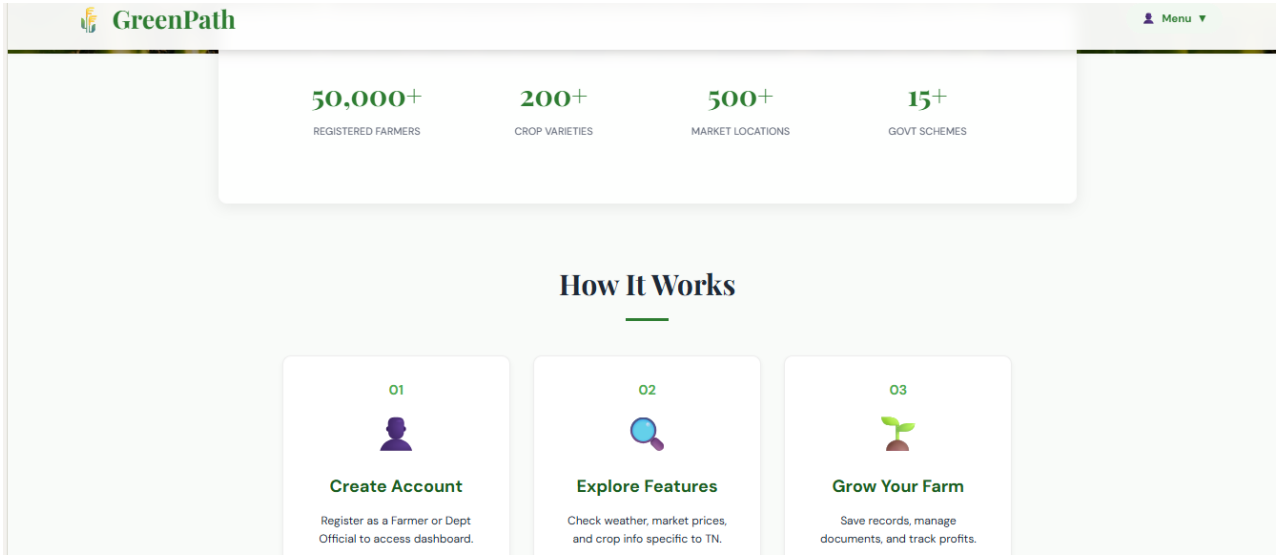
9.1 Home Page

Displays the main landing interface of the GreenPath Agriculture Information System, introducing the platform to farmers.



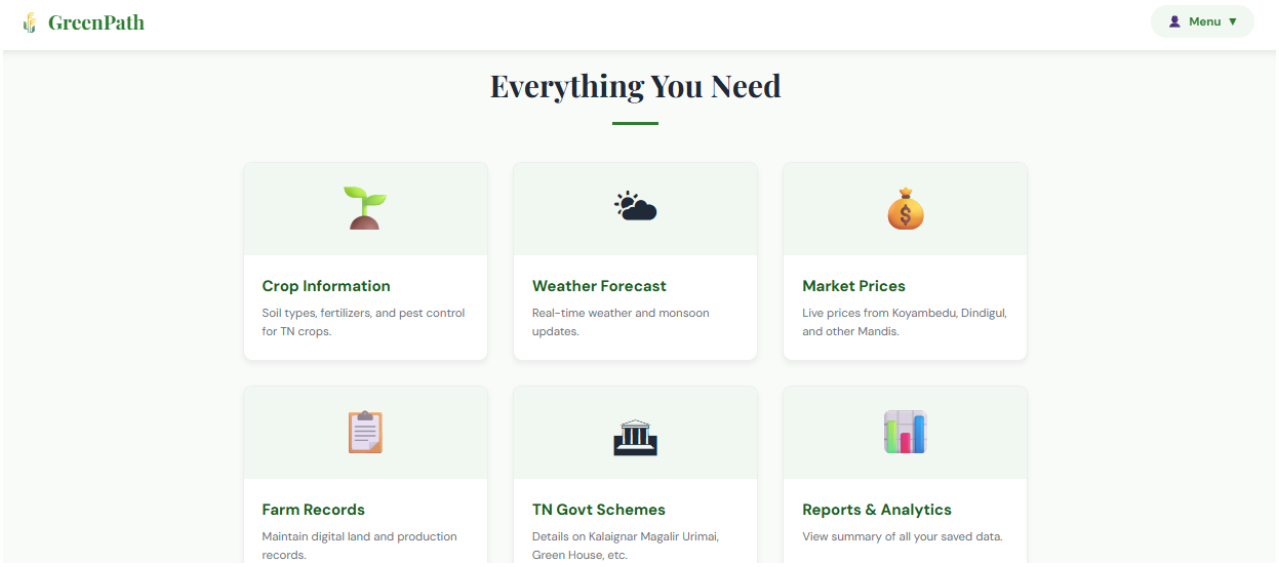
9.2 Platform Statistics & How It Works

Shows key platform statistics (registered farmers, crop varieties, market locations, government schemes) and the three-step onboarding process.



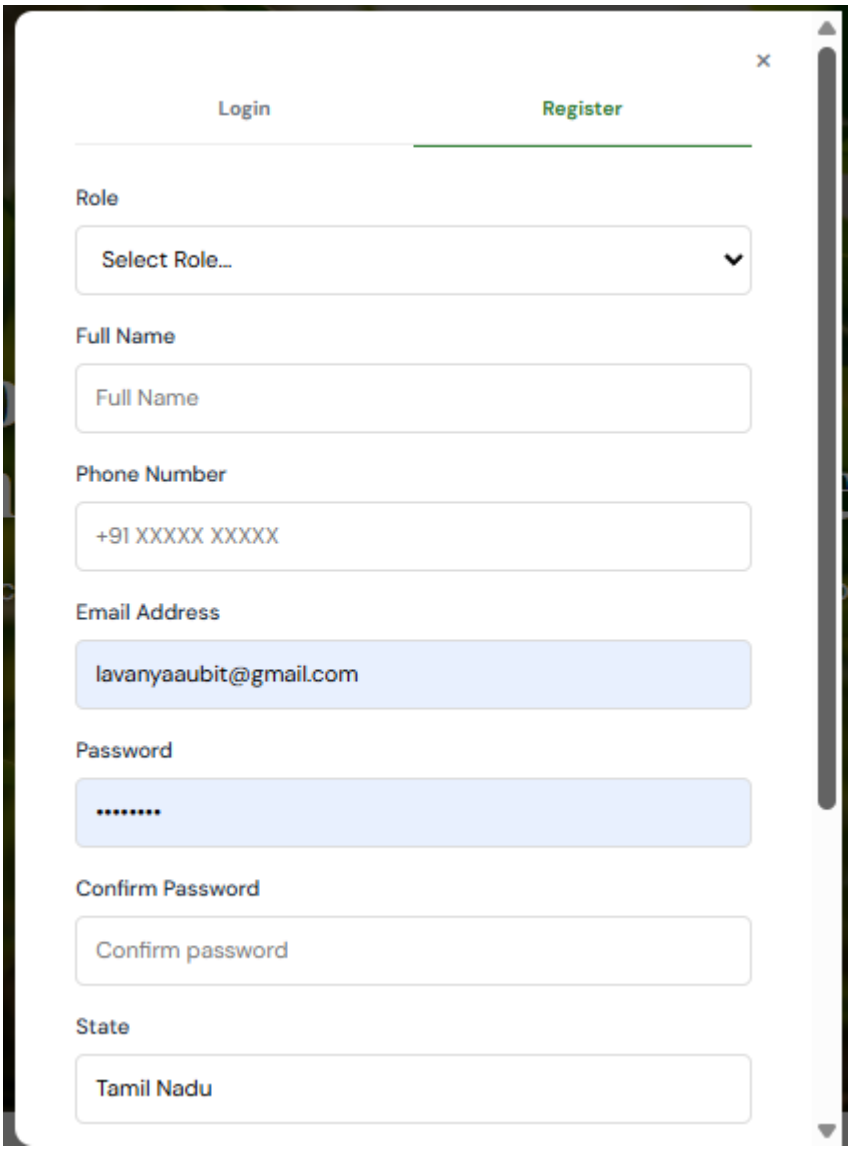
9.3 Features Overview

Displays all the core modules available to users — Crop Information, Weather Forecast, Market Prices, Farm Records, TN Govt Schemes, and Reports.



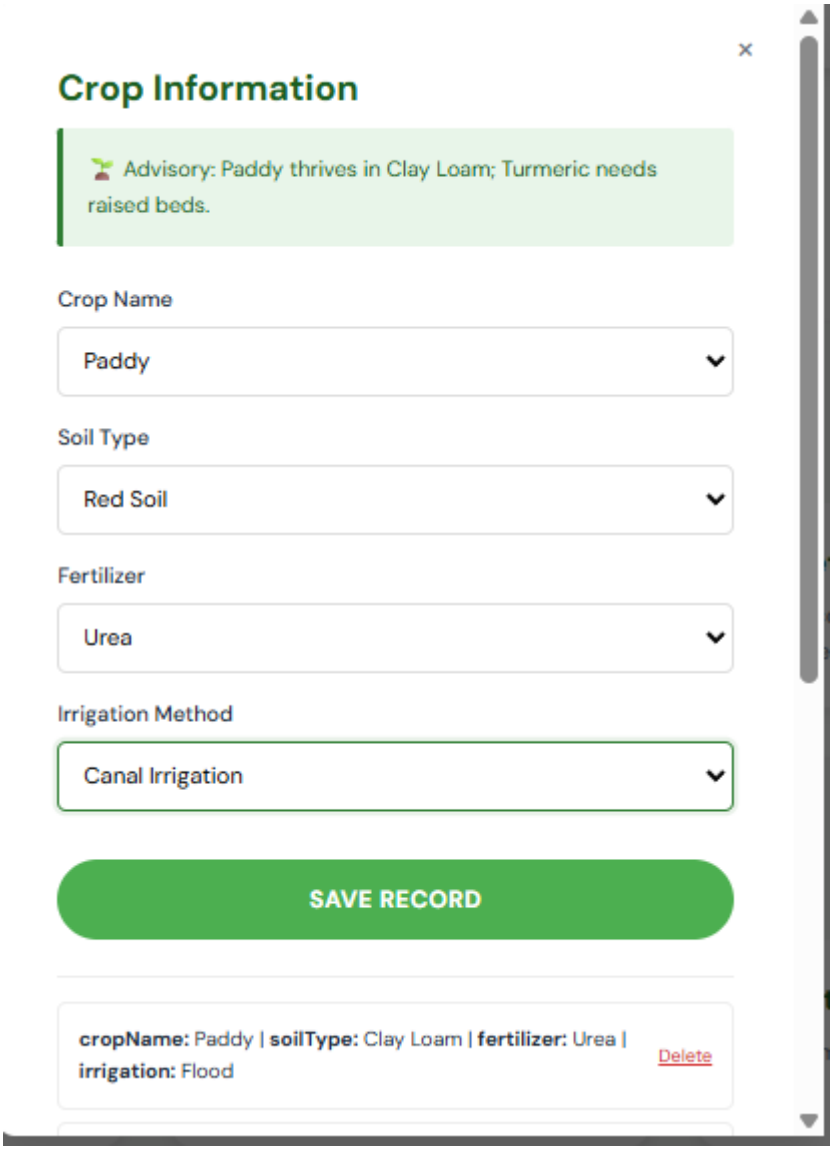
9.4 Registration Page

Allows new users (Farmer or Department Official) to create an account by providing their role, name, phone number, email, password, and state.



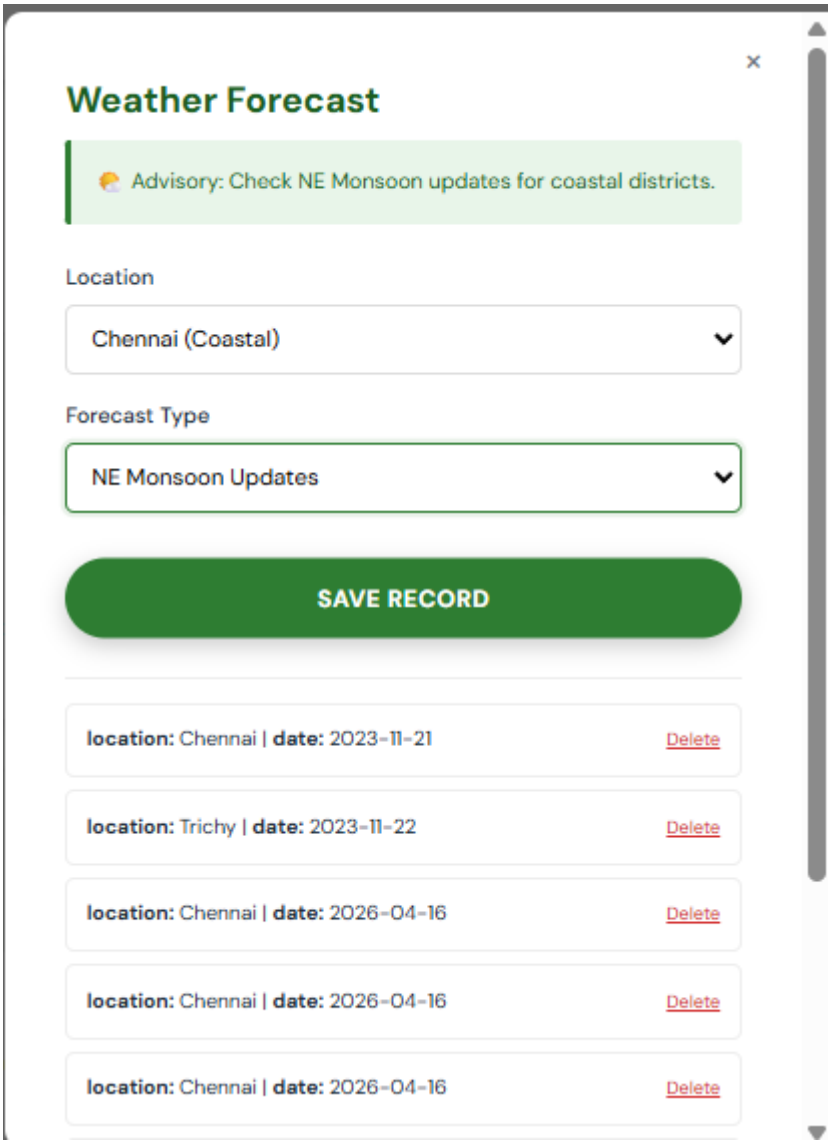
9.5 Crop Information Module

Lets farmers select a crop, soil type, fertilizer, and irrigation method, and displays crop-specific advisory messages along with previously saved records.



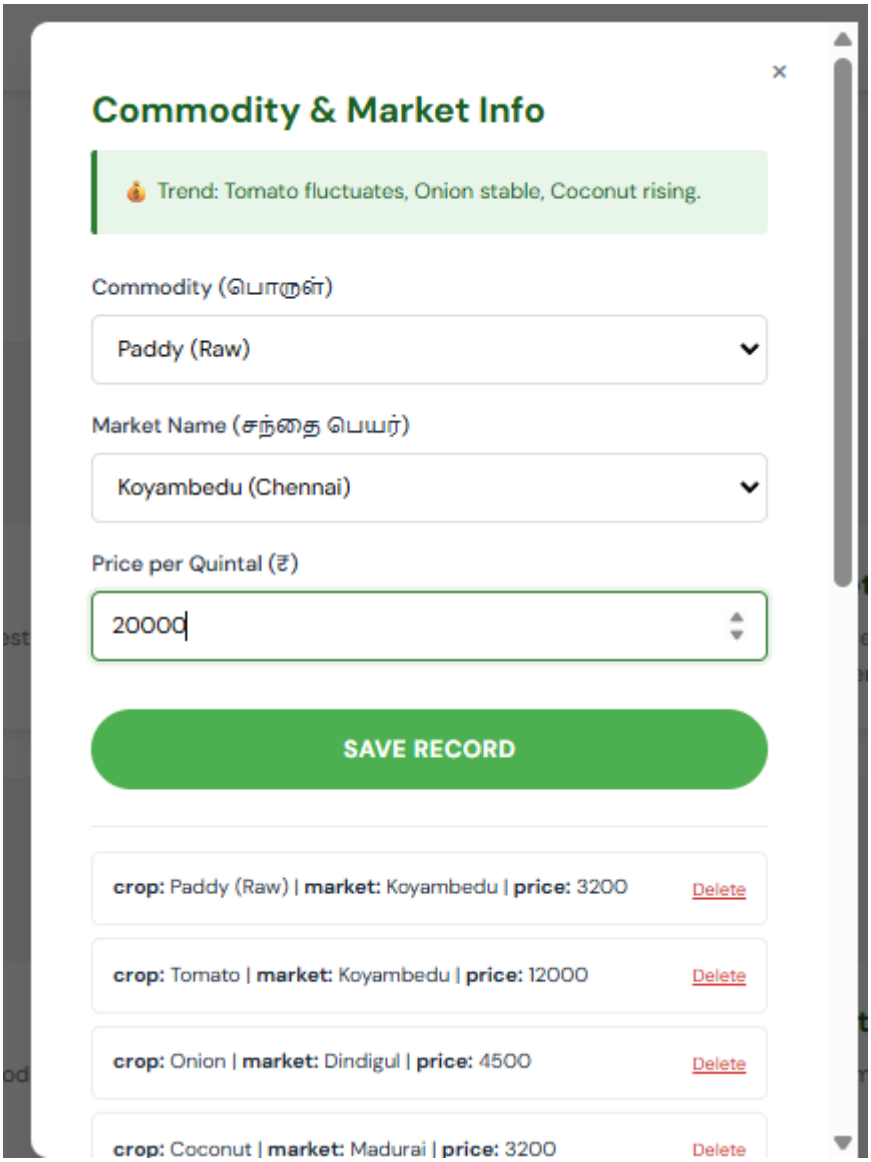
9.6 Weather Forecast Module

Provides location-based weather and monsoon forecast information along with advisory alerts and a history of saved forecast records.



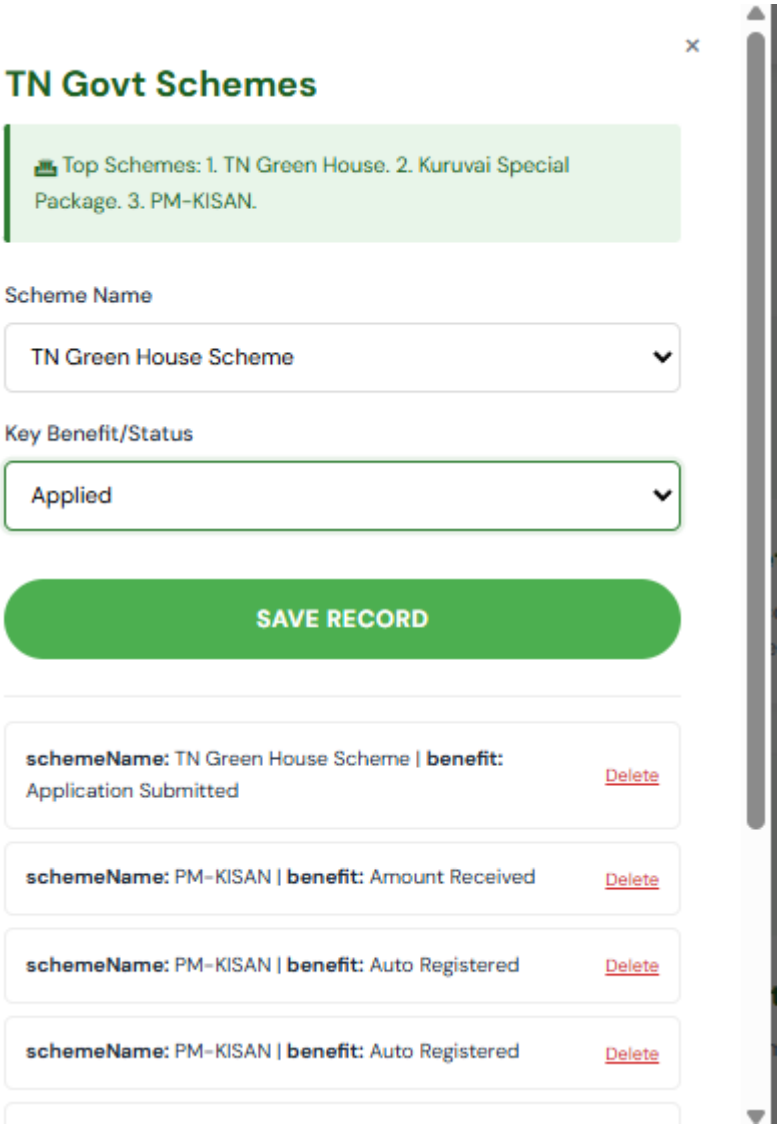
9.7 Commodity & Market Prices Module

Displays live market price trends for various commodities across different market locations, with the ability to save and view price records.



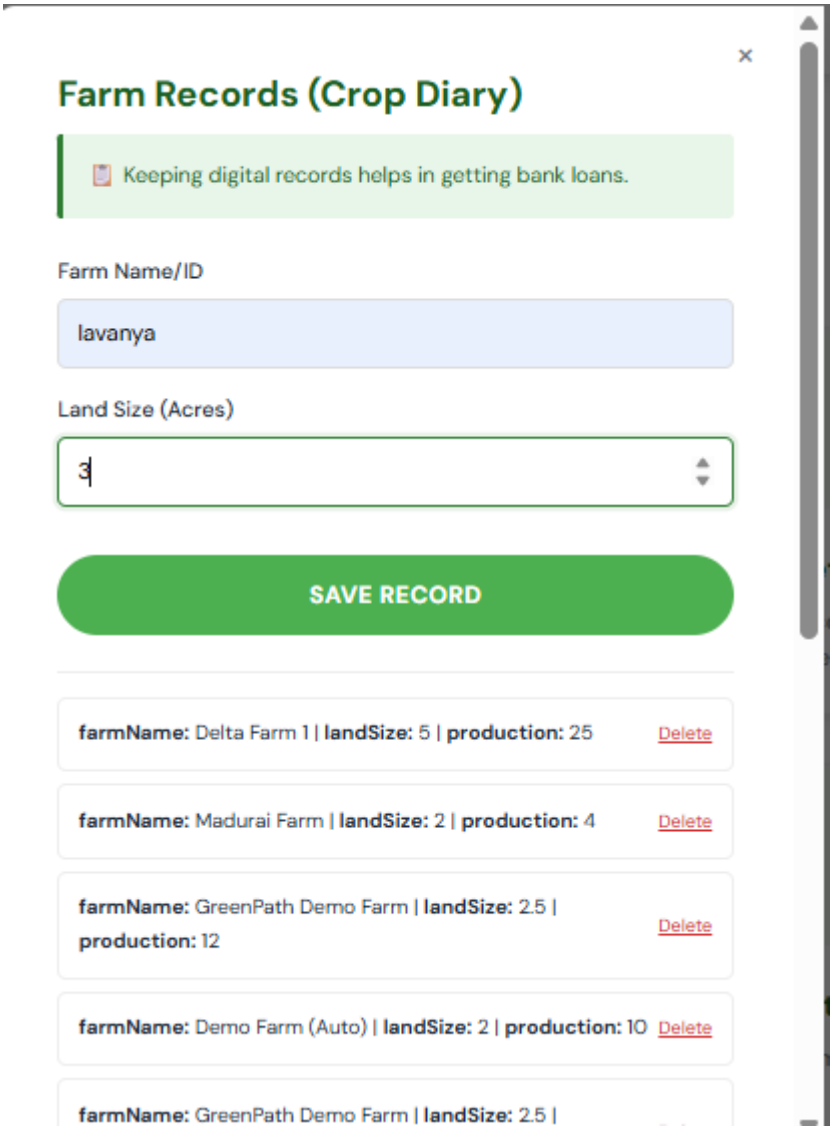
9.8 TN Government Schemes Module

Lists key Tamil Nadu government schemes for farmers along with benefit/application status tracking, such as the TN Green House Scheme and PM-KISAN.



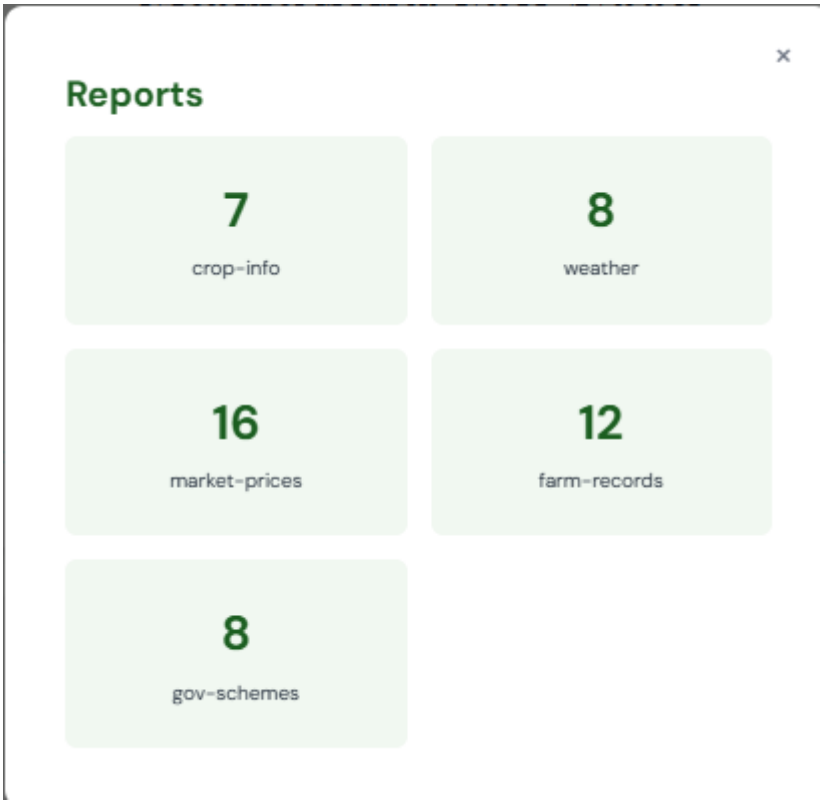
9.9 Farm Records (Crop Diary) Module

Enables farmers to maintain digital land and production records for each farm, helping with tasks such as bank loan applications.



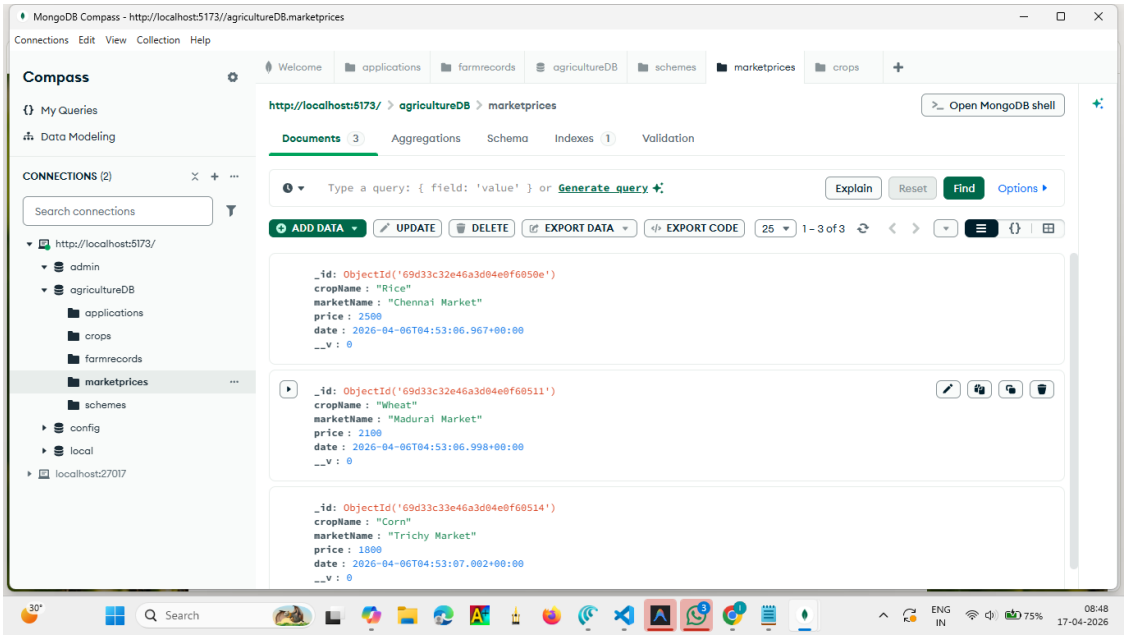
9.10 Reports Dashboard

Summarizes the total number of saved records across all modules — crop information, weather, market prices, farm records, and government schemes.



9.11 Database – MongoDB Compass (Market Prices Collection)

Shows the underlying MongoDB collection 'marketprices' within the 'agricultureDB' database, storing crop name, market name, price, and date fields.



10. ANALYSIS

Testing of the Agriculture Information System was carried out module by module to validate both functional correctness and overall system behaviour.

Functional analysis confirmed that registration, login, crop information lookup, weather forecast retrieval, market price browsing, farm record management, and government scheme browsing all behaved as expected under normal and edge-case inputs (e.g., duplicate emails, invalid credentials, empty fields).

Performance observations showed that REST API response times remained fast for typical operations due to the lightweight nature of the Express server and MongoDB's flexible document model; response times are expected to remain predictable as the crops, marketprices, and farmrecords collections grow.

Usability analysis indicated that the React-based, card-driven responsive design adapted well across desktop and mobile screen sizes, and the feature-grid layout on the home page reduced the number of steps a farmer needs to reach the information they want, compared to informal, word-of-mouth channels.

Comparative analysis against the existing manual approach highlighted clear improvements: the time to check a crop's soil and fertilizer guidance, or a market's current price, was reduced from an in-person or phone-based inquiry to a few clicks; farm record-keeping moved from paper (or nothing at all) to a searchable digital record; and access to government scheme information became centralized rather than dependent on informal networks.

Overall, the analysis confirms that the system meets its stated objectives of centralizing agricultural information, reducing the effort needed to access it, and providing a more efficient, accessible platform for farmers and department officials alike.

11. CONCLUSION

The Agriculture Information System successfully demonstrates how full-stack web technologies can be applied to give farmers centralized, timely access to the information they need – crop and soil guidance, weather forecasts, market prices, farm records, and government schemes. By replacing scattered, informal sources with a centralized digital platform, the system reduces the effort required to find reliable agricultural information and supports better-informed farming decisions.

The project's layered architecture – comprising a responsive React frontend and a Node.js/Express backend backed by MongoDB – proved effective in cleanly separating concerns while enabling smooth communication through REST APIs. Core modules including user authentication, crop

information, weather forecasting, market prices, farm records, and government schemes were implemented and tested successfully, validating the system's practical applicability.

Beyond the technical deliverable, this project provided valuable hands-on experience in full-stack (MERN-style) development, REST API design, MongoDB schema design, and structured software testing – skills directly applicable to real-world software development, and to building technology that can genuinely support the agricultural community.

12. FUTURE WORK

- Integration of AI-based crop and fertilizer recommendations tailored to soil and location data
- Automated SMS/email alerts for weather warnings, price changes, and new government schemes
- Development of a dedicated mobile application for Android and iOS
- Multilingual support, including Tamil, to widen accessibility for local farmers
- Integration with live government scheme APIs for automatically updated scheme listings
- Cloud deployment (AWS/Azure/GCP) for improved scalability and availability
- Advanced analytics and dashboards summarising regional crop and price trends
- Offline/low-connectivity support for use in areas with limited internet access
- Role-based access control with JWT-based authentication for enhanced security

13. REFERENCES

1. MDN Web Docs – JavaScript, HTML, and CSS reference documentation.
2. React Official Documentation – Components, hooks, and frontend development.
3. Node.js Official Documentation – Runtime and core module reference.
4. Express.js Official Documentation – Routing and REST API development.
5. MongoDB Official Documentation – Database design and Mongoose ODM usage.
6. GeeksforGeeks – MERN stack concepts and programming examples.